

• Sec-A

1. Explain the use of else in loops?

Ans in loops, the 'else' statement is used to execute a block of code when the loop completes normally without encountering a "break" statement. This is often used for executing some finalization code or handling a specific condition after the loop has iterated through all its items. If the loop is terminated early with a "break" statement, the 'else' block is not executed.

eg:

```
numbers = [1, 3, 5, 7]
```

```
search_element = 8
```

```
for number in numbers:
```

```
    if number == search_element:
```

```
        print("Found", search_element)
```

```
        break
```

```
    else:
```

```
        print(search_element, "not found in the list")
```

2. What is the pass statement in Python?

Ans The pass statement in Python is a null statement, meaning it does nothing when executed. It's primarily used as a placeholder in various contexts where the syntax requires a statement but you don't want any actual code to run at that point. And 'pass' is a keyword, not a function.

Here are some common use cases for the 'pass' statement.

- (i) placeholder
- (ii) Empty blocks
- (iii) Unimplemented functionality

3. What is slice operation? Explain with an example.

Ans The slice operation in python is a powerful way to extract a specific portion of a sequence like a string, list, or tuple. It uses a concise syntax with square brackets `[]` to create a subsection of the original sequence.

e.g:

```
= original_list = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
subset = original_list[2:5]
print(subset)
```

output = 2, 3, 4

4. Explain the concept of range function in python.

Ans The `range()` function in python is a versatile tool for generating sequences of numbers. It creates an immutable sequence of integers, meaning you cannot modify the generated sequence after creation.

Syntax:

```
range(start, stop, step)
```

e.g: (i) for num in range(5):

```
print(num)
```

output = 0, 1, 2, 3, 4

(ii)

```
(ii) for num in range(2, 7):
```

```
print(num)
```

output = 2, 3, 4, 5, 6

```
(iii) for num in range(1, 11, 2):
```

```
print(num)
```

output = 1, 3, 5, 7, 9

5. Which conditional operators are supported in python?

Ans: Python supports several conditional operators for comparisons. These operators are used to evaluate conditions and produce boolean results. Here are main conditional operators supported in python:

- (i) Equal to: '`==`'
- (ii) Not equal to: '`!=`'
- (iii) Less than: '`<`'
- (iv) Less than or equal to: '`<=`'
- (v) Greater than: '`>`'
- (vi) Greater than or equal to: '`>=`'

• Sec-B ($3 \times 7 = 21$)

1. What is the ^{difference} statement betⁿ break and continue statement in loop?

Ans: Both break and continue statements are used to alter the flow of control within loops in Python, but they serve different purposes:

(i) Break statement:

• Function: Exits the loop immediately, even if there are remaining iterations.

• Use case: When you want to terminate the loop prematurely upon encountering a specific condition.

eg: `numbers = [1, 2, 3, 4, 5]`

`for number in numbers:`

`if number > 3:`

`break`

`print(number)`

output:

1

2

3

(ii) Continue Statement:

- Function: Skips the current iteration of the loop and moves directly to the next iteration.
- Use cases: When you want to skip specific iterations based on a condition and continue processing the remaining ones.

e.g.:

```

numbers = [1, 2, 4, 5]
for num in numbers:
    if num % 2 == 0:      # skip even no.
        continue
    print(num)

```

output: 1
5

• Key differences:

Features	break statement	Continue Statement
• Action	Exist the loop entirely	Skip the current iteration
• Remaining iterations	Not processed	Processed after the current iteration is skipped.
• Use case	Permanently terminate the loop.	SKIP specific iterations based on a condition.

2. Write a Python Program to access the last character of the string (a) with and (b) without the help of len() function.

Ans

(a) With the help of 'len()' function:

a = "Ankit"

last_character = a[len(a)-1]

print("Last character (with len()):", last_character)

Output: Last character (with len()): t

(b) without using the 'len()' function:

```
a = "Ankit"
last_character = a[-1]
print("Last character (without len()):", last_character)
```

output: Last character (with len()): t

2. Explain the various methods to test if the entered string contain digits or alphabets or alphanumeric values.

Ans To test if an entered string contains digits, alphabets, or alphanumeric values, you can use various methods and functions available in Python. Here are some common approaches:

- (i) using 'str.isdigit()', 'str.isalpha()', and 'str.isalnum()' methods:
- 'str.isdigit()': Return 'True' if all char. in the string are digits.
 - 'str.isalpha()': Return 'True' if all char. in the string are alphabetic.
 - 'str.isalnum()': Return 'True' if all char. in the string are alphanumeric.

e.g.:

```
entered string = input("Enter a string: ")
s = input("Enter a string: ")
if s.isdigit():
    print("Entered string contains only digits.")
elif s.isalpha():
    print("Entered string contains only alphabets.")
elif s.isalnum():
    print("Entered string contains alphanumeric values.")
else:
    print("Entered string contains other characters.")
```

Output: Enter a string: ankit
Enter string contains only alphabets.

(ii) Using Regular Expressions ('re' module):

- Regular expressions provide powerful pattern matching capabilities.
- We can use the 're.match()' fun. to match pattern in the string.

e.g:

```
import re
s = input("Enter a string:")
if re.match("[0-9]+$", s):
    print("String contains only digits.")
elif re.match("[a-zA-Z]+$", s):
    print("String contains only alphabets.")
elif re.match("[a-zA-Z0-9]+$", s):
    print("String contains alphanumeric values.")
else:
    print("Entered string contains other characters.")
```

Part - C (11*3=33)

1. (A) WAP to remove all the characters from a string except integers.

Ans

Program:

```
input_string = input("Enter a string:")
result_string = ''
for char in input_string:
    if char.isdigit():
        result_string += char
print("String with only integers:", result_string)
```

Output: Enter a string: ankit1068
String with only integers: 1068

1(B) WAP to remove special symbols/punctuation from a string.

Ans

```
input_string = input("Enter a string:")
result_string = ''
```

```
for char in input_string:
    if char.isalnum() or char == ' ':
        result_string += char
```

```
print("String without special symbol/punctuation:", result_string)
```

Output: Enter a string: ank1t@#5%^&*?!.

String without special symbol/punctuation: ank1t

Q 2. WAP to print following pattern:

```
1
1 2 1
1 2 4 2 1
1 2 4 8 4 2 1
1 2 4 8 16 8 4 2 1
1 2 4 8 16 32 16 8 4 2 1
1 2 4 8 16 32 64 32 16 8 4 2 1
1 2 4 8 16 32 64 128 64 32 16 8 4 2 1
```

Ans

rows = 8

```
for i in for i in range(1, rows + 1):
```

```
    for j in range(1, i + 1):
```

```
        print(2**(j-1), end = ' ')
```

```
    for j in range(i-1, 0, -1):
```

```
        print(2**(j-1), end = ' ')
```

```
    print("")
```


output:

```

1
1 2 1
1 2 4 2 1
1 2 4 8 4 2 1
1 2 4 8 16 8 4 2 1
1 2 4 8 16 32 16 8 4 2 1
1 2 4 8 16 32 64 32 16 8 4 2 1
1 2 4 8 16 32 64 128 64 32 16 8 4 2 1

```

Q3. (A) WAP to count digit in a given number.

Ans

```

number = int(input("Enter a number:"))
count = 0

```

```

for char in str(number):

```

```

    if char.isdigit():

```

```

        count += 1

```

```

print("Number of digits in the given number:", count)

```

output: Enter a number: 12345678943

Number of digit in the given number : 11

Q 2. (B) Explain the Control statements in python in detail.

Ans

Control statements are fundamental building block in any programming language, and python is no exception. They allow you to control the flow of execution in your code, making decision and performing actions based on certain conditions.

There are three main types of Control Statement in python:

(i) Conditional statement (if, elif, else):-

Conditional statement allow you to execute certain blocks of code based on whether a specified condition is true

or false. The main Conditional statements in python are:

- If statement: Executes a block of code if a specified condition is true.
- elif statement: Stands for "else if". Allows you to check multiple conditions after the initial "if" statement. If the condition in the "if" statement is false, it checks the condition in the "elif" statement.
- else statement: Executes a block of code if none of the preceding conditions are true.

eg:

```
x = 10
if x > 10:
    print("x is greater than 10")
elif x < 10:
    print("x is less than 10")
else:
    print("x is equal to 10")
```

(ii) Looping statements (for, while): Looping statements allow you to execute a block of code repeatedly as long as a specified condition is true. The main looping statements in python are:

- For loop: Executes a block of code a specified number of times or iterates over items in a sequence (e.g. lists, tuples, strings).
- While loop: Executes a block of code as long as a specified condition is true. It repeatedly checks the condition before executing the block of code.

eg. of (for loop):-

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

Example (While loop):

```
i = 0
while i < 5:
    print(i)
    i += 1
```

(iii) Control Flow statements (break, continue, pass):

Control flow statements allow you to control the flow of execution within loops or conditional statements.

- **break statement:** Terminates the nearest enclosing loop.
- **Continue statement:** skips the rest of the current iteration of a loop and proceeds to the next iteration.
- **pass statement:** Does nothing. it acts as a placeholder when a statement is syntactically required but no action is needed.

eg.

```
for i in range(10):
```

```
    if i == 3:
```

```
        break
```

```
    elif i == 1:
```

```
        continue
```

```
    else:
```

```
        pass
```

```
    print i
```

Output

0

2

← The end

(22BCON1068)